

Tubería CNN en OpenACC

Convolución 2D + ReLU + MaxPooling 2D

Documentación Técnica y Reporte de Rendimiento

Juferoga

Computación Paralela y Distribuida

Maestría en Ciencias de la Información y las Comunicaciones (MCIC)

Universidad Distrital Francisco José de Caldas

Septiembre de 2026

Resumen

Este documento presenta el diseño, implementación y evaluación experimental de una tubería de operaciones fundamentales de Redes Neuronales Convolucionales (Convolución 2D de $100 \times 100 \rightarrow 98 \times 98$, activación ReLU y MaxPooling 2D con Stride 2 hacia 49×49) acelerada en GPU mediante directivas OpenACC en C++. Se aborda la contención de datos en VRAM a lo largo de ráfagas iterativas ($N = 1\,000$ iteraciones) mediante la directiva `#pragma acc data`, la maximización de la ocupación de los multiprocesadores CUDA mediante `collapse(2)`, la corrección teórica formal sobre la naturaleza topológica de la ventana de MaxPooling, y la evaluación cuantitativa frente a una línea base secuencial pura en CPU, alcanzando una aceleración de cómputo neto de **15.45x** y de sistema completo de **13.08x** con verificación numérica exacta ($\Delta_{\max} = 0.000$).

Índice

1. Introducción y Arquitectura de la Tubería	2
2. Clarificación Conceptual y Corrección Teórica	2
3. Mapeo Canónico 1D (Row-Major Order)	2
4. Estrategia de Paralelización en OpenACC	2
4.1. Ciclo de Vida de Datos y Persistencia en VRAM	2
4.2. Colapso de Bucles Bidimensionales	3
5. Resultados Experimentales y Ley de Amdahl	3
6. Conclusiones	4

1. Introducción y Arquitectura de la Tubería

En las capas convolucionales modernas, el procesamiento secuencial e iterativo de tensores espaciales demanda un uso intensivo de cómputo aritmético y ancho de banda de memoria. La secuencia estudiada en este reto consta de tres etapas fijas:

1. **Convolución 2D:** Aplica un kernel espacial de 3×3 sobre una imagen de entrada 100×100 (10 000 flotantes), generando una matriz de características de 98×98 (9 604 flotantes).
2. **Activación ReLU:** Transforma no linealmente cada elemento de la matriz anterior preservando su tamaño (98×98), sustituyendo cualquier valor negativo por cero ($f(x) = \max(0, x)$).
3. **MaxPooling 2D:** Realiza un submuestreo espacial disjunto mediante una ventana de 2×2 con paso (*stride*) de 2, produciendo una salida final compacta de 49×49 (2 401 flotantes).

2. Clarificación Conceptual y Corrección Teórica

A diferencia del filtro convolucional (que sí posee una matriz de parámetros entrenables 3×3), la “ventana de MaxPooling” **no es una matriz de pesos ni requiere asignación de memoria dedicada en RAM/VRAM**. Corresponde estrictamente a un **rango de lectura e indexación espacial**:

$$\text{pool}(i, j) = \max_{(k_i, k_j) \in \{0,1\}^2} \text{relu}(2i + k_i, 2j + k_j) \quad (1)$$

La corrección teórica evita destinar memoria espuria o simular operaciones de convolución en el paso de submuestreo.

3. Mapeo Canónico 1D (Row-Major Order)

Para optimizar el uso de cachés y habilitar accesos coalescentes en la memoria global de la GPU, todas las matrices bidimensionales se asignan en memoria dinámica contigua (`new float [N]`):

- **Imagen de Entrada (100×100):** $\text{idx} = i \times 100 + j$, rango $[0 \dots 9\,999]$.
- **Buffer Convolución y ReLU (98×98):** $\text{idx} = i \times 98 + j$, rango $[0 \dots 9\,603]$.
- **Salida MaxPooling (49×49):** $\text{idx} = i \times 49 + j$, rango $[0 \dots 2\,400]$.

Para MaxPooling con Stride 2 ($W = 98$), las 4 direcciones de lectura contigua son:

$$\begin{aligned} r_{00} &= (2i) \cdot 98 + 2j, & r_{01} &= (2i) \cdot 98 + (2j + 1) \\ r_{10} &= (2i + 1) \cdot 98 + 2j, & r_{11} &= (2i + 1) \cdot 98 + (2j + 1) \end{aligned}$$

4. Estrategia de Paralelización en OpenACC

4.1. Ciclo de Vida de Datos y Persistencia en VRAM

El bus PCIe impone una latencia crítica en sistemas acelerados. Para evitar transferencias redundantes en cada iteración, se emplea la directiva estructurada `#pragma acc data`:

```
#pragma acc data copyin(img[0:10000], kernel[0:9]) \
                    create(conv[0:9604], relu[0:9604]) \
                    copyout(pool[0:2401])
{
    for (int it = 0; it < iteraciones; ++it) {
        // Convolucion 2D (GPU) -> conv en VRAM
        // Activacion ReLU (GPU) -> relu en VRAM
        // MaxPooling 2D (GPU) -> pool en VRAM
    }
}
```

- **copyin:** Transfiere entrada y kernel a la GPU exactamente una vez al inicio.
- **create:** Aloja los buffers temporales conv y relu en VRAM con **cero tráfico PCIe**.
- **copyout:** Devuelve al Host únicamente la salida final compacta al terminar la ráfaga de 1,000 repeticiones.

4.2. Colapso de Bucles Bidimensionales

Cada etapa fusiona sus bucles anidados mediante `collapse(2)`, expandiendo el paralelismo para saturar los multiprocesadores CUDA:

- Convolución 2D: $98 \times 98 = 9\,604$ hilos GPU.
- Activación ReLU: $98 \times 98 = 9\,604$ hilos GPU.
- MaxPooling 2D: $49 \times 49 = 2\,401$ hilos GPU.

5. Resultados Experimentales y Ley de Amdahl

A continuación se resumen las mediciones obtenidas para $N = 1\,000$ iteraciones:

Cuadro 1: Métricas de Rendimiento Experimental ($N = 1\,000$ iteraciones)

Métrica de Evaluación	Valor Medido	Unidad
CPU Secuencial (T_{CPU})	32.46	ms
GPU Cómputo Neto ($T_{\text{GPU_Kernel}}$)	2.10	ms
GPU Total con Transferencias ($T_{\text{GPU_Total}}$)	2.48	ms
Overhead de Bus PCIe	0.38	ms (15.3 %)
Speedup de Cómputo ($S_{\text{cómputo}}$)	15.45x	Aceleración
Speedup Total de Sistema (S_{total})	13.08x	Aceleración
Consistencia Numérica ($\Delta_{\text{máx}}$)	0.000	0 Discrepancias (2 401/2 401)

Análisis de Amdahl: Si se ejecutase una sola iteración ($N = 1$), el costo de transferencias PCIe (0.38 ms) superaría con creces los microsegundos de cómputo neto, arrojando una desaceleración ($S < 1\times$). La ejecución en ráfaga iterativa amortiza el costo de transferencia a $0.38\ \mu\text{s}$ por iteración, posibilitando una aceleración real de más de **13x**.

6. Conclusiones

1. OpenACC permite acelerar algoritmos de visión artificial y redes neuronales manteniendo una base de código compacta y legible en C++ estándar.
2. La gestión explícita de datos con `#pragma acc data` y la cláusula `create` es el pilar determinante de la eficiencia en arquitecturas heterogéneas.
3. La verificación numérica bit a bit valida que la paralelización masiva de las etapas preserva de forma estricta la equivalencia funcional del modelo secuencial.